Code Running Guide

How to Run and Extend the Note 2 Python Examples

Installation, smoke tests, training diagnostics, outputs, and troubleshooting

Companion to the PINN/PIDL cyber-control tutorial note

Contents

1. What This Guide Covers

This guide is a standalone manual for the Python examples in Note 2. It explains how to set up the environment, run the fast checks, run longer PINN/PIDL diagnostics, interpret the generated outputs, and extend the examples to richer cyber-control models.

2. Folder Structure

The repository is organized around four teaching experiments:

```
.
  requirements.txt
  README.md
  README.md
  scripts/
    run_smoke_tests.sh
    generate_figures.py
    run_training_iterations.py
  src/
    inverse_pinn_sir_malware.py
    pidl_unknown_mechanism.py
    control_pinn_malware.py
    pmp_informed_pinn_malware.py
  artifacts/experiments/
    OUTPUT_PREVIEW.md
    training_summary.md
    baseline_comparison_metrics.csv
    inverse_pinn_training_history.csv
    pidl_training_history.csv
    control_pinn_training_history.csv
    pmp_informed_pinn_training_history.csv
  docs/assets/
    inverse_pinn_sparse_data.png
    pidl_missing_mechanism.png
    neural_architectures.png
    training_iteration_diagnostics.png
    baseline_comparison.png
```

3. Installation

Create a Python virtual environment and install dependencies:

```
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install --upgrade pip
pip install -r requirements.txt
```

The examples require NumPy, PyTorch, and Matplotlib. CUDA is optional; the checked-in runs are

CPU-friendly.

4. Quick Smoke Test

Run the fast local check:

```
bash scripts/run_smoke_tests.sh
```

A smoke test checks syntax and basic data flow. It does not train high-accuracy PINNs or prove identifiability.

5. Normal Runs

5.1 Static figure generation

```
python scripts/generate_figures.py
```

Expected output: refreshed PNG figures under `docs/assets/`, including sparse-data setup, PIDL missing-mechanism, and neural-architecture diagrams.

5.2 Inverse PINN

```
python src/inverse_pinn_sir_malware.py --iters 1000
```

Expected output: total loss, learned beta/gamma, data loss, ODE residual loss, and initial-condition loss.

5.3 PIDL missing mechanism

```
python src/pidl_unknown_mechanism.py --iters 1000
```

Expected output: total loss, data loss, residual loss, correction regularizer, and mean learned correction.

5.4 Direct control PINN

```
python src/control_pinn_malware.py --iters 1000
```

Expected output: objective proxy, state residual, initial-condition loss, and mean control intensity.

5.5 Heterogeneous node-SIPRS inverse PINN

```
python src/node_siprs_inverse_pinn.py --smoke --device cpu
```

Expected output: held-out state MSE, held-out-node state MSE, homogeneous-misspecification rollout MSE, and community susceptibility, infectivity, and recovery RMSE. The smoke case fixes the base beta and learns a low-dimensional community-rate map rather than one global beta/gamma for heterogeneous truth.

5.6 PMP-informed PINN

```
python src/pmp_informed_pinn_malware.py --iters 1000
```

Expected output: state residual, costate residual, stationarity residual, and boundary residual.

5.7 Longer training diagnostics

```
python scripts/run_training_iterations.py --iters 600
```

Expected output:

- CSV histories under `artifacts/experiments/`;
- baseline metrics in `artifacts/experiments/baseline_comparison_metrics.csv`;
- `OUTPUT_PREVIEW.md` and `training_summary.md`;
- `artifacts/figures/training_iteration_diagnostics.png` and `artifacts/figures/baseline_compari`

6. How to Interpret Outputs

Output	Interpretation
Data loss	How well the learned state matches observed data. Low data loss alone does not prove the dynamics are right.
ODE residual loss	How well the neural derivative satisfies the differential equation at collocation points.
Initial-condition loss	Whether the learned trajectory starts from the prescribed cyber state.
Correction regularizer	Whether the PIDL correction remains controlled instead of replacing the known mechanism.
Objective proxy	The direct-control PINN's malware-control objective. It should be checked together with residual loss.
Stationarity loss	Whether the PMP-informed control satisfies the Hamiltonian stationarity condition.
Boundary loss	Whether the state initial condition and costate terminal condition are satisfied.

7. Output Map

Start with `artifacts/experiments/OUTPUT_PREVIEW.md`. It groups the generated results by experiment, diagnostic panel, baseline-comparison panel, and file. Then inspect `artifacts/figures/training_iteration_d` `artifacts/figures/baseline_comparison.png`, and the CSV file for the experiment you are studying.

File	What it shows
`inverse_pinn_training_history.csv`	Total, data, ODE, and initial-condition losses plus learned beta/gamma.
`pidl_training_history.csv`	Known-mechanism residual loss, correction regularization, and mean learned correction.

`control_pinn_training_history.csv`	Objective, state residual, initial-condition loss, and mean control.
`pmp_informed_pinn_training_history.csv`	State, costate, stationarity, and boundary residuals.
`node_siprs_inverse_history.csv`	Held-out node-state error, mass error, and community-rate RMSE for the heterogeneous graph inverse-PINN smoke route.
`baseline_comparison_metrics.csv`	Topic-specific baseline metrics for inverse PINN, PIDL, and controlled malware mitigation.
`training_iteration_diagnostics.png`	Four-panel comparison of the main loss and residual signals.
`baseline_comparison.png`	Learned methods compared with sparse interpolation, wrong-parameter SIR, known-SIR-only dynamics, and no/fixed-control baselines.

8. Extending the Code

8.1 From sparse inverse PINN to noisy cyber data

Add observation noise, hold out a validation set, and report parameter uncertainty. Avoid claiming exact parameter recovery unless the data are synthetic and the ground truth is known.

8.2 From PIDL correction to richer missing mechanisms

Change the correction network input from (t, x) to (t, x, u) or to graph features when the missing mechanism depends on interventions or network structure. Keep a correction regularizer so the learned term remains interpretable.

8.3 From open-loop to feedback neural control

Change the control network input from t to (t, x), then train over multiple initial states and parameter scenarios. This produces a feedback-like neural control law rather than one open-loop control curve.

8.4 From small SIR models to network-scale PINNs

Move gradually: first degree-level compartments, then node-level vectors or graph-neural PINNs. For stiff, impulsive, or hybrid dynamics, use piecewise networks and add collocation points near jump times.

9. Troubleshooting

- If data loss is low but residual loss is high, increase residual weight or add collocation points.
- If residual loss is low but prediction is poor, the assumed model may be wrong or parameters may be unidentifiable.
- If PIDL correction dominates the known mechanism, increase correction regularization or reduce correction-network width.
- If direct-control PINN lowers the objective but violates dynamics, increase state-residual weight and

validate the learned control with an independent ODE solver.

- If PMP-informed stationarity loss remains high, check control bounds and consider projected or KKT-style stationarity residuals.

10. Reproducibility Checklist

For every experiment, record:

- model equations and parameter values;
- random seed, initial states, and observation schedule;
- collocation-point sampling strategy;
- network architecture and optimizer;
- loss weights for data, residual, boundary, objective, and PMP terms;
- number of iterations and logging interval;
- all baseline solvers or held-out checks used for comparison.